



ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia
a informatiky

Semestrálna práca z predmetu
vývoj aplikácií pre mobilné zariadenia

EXPENSE SPLITTER

Vypracoval: Peter Kása

Študijná skupina: 5ZYI23

Akademický rok: 2025/26

V Žiline dňa 24.5.2026



Obsah

Úvod	2
1. Analýza Riešenia	2
Model (Dátová vrstva):	2
View (Používateľské rozhranie):	2
ViewModel (Logická vrstva):	2
Rozdelenie Balíkov.....	2
2. Návrh Riešenia.....	3
Možnosti pre používateľa.....	3
Diagram pridania výdavku	3
3. Implementácia.....	3
Ošetrovanie rotácie.....	3
Navigácia	3
Sieťová komunikácia a Offline režim	3
Real-time Update	3
Matematická validácia.....	3
Bezpečnosť a Exception Handling	3
Notifikácie.....	4
4. Zdroje	4



Úvod

Cieľom tejto semestrálnej práce bolo vytvorenie mobilnej aplikácie, ktorá slúži na správu spoločných výdavkov medzi viacerými používateľmi a uľahčiť študentom a skupinám priateľov správu spoločných výdavkov. Aplikácia umožňuje používateľom vytvárať skupiny, zaznamenávať výdavky a automaticky vypočítať, kto je komu dlžný. Mojou motiváciou pre vytvorenie tejto aplikácie je problém pri zdieľaní nákladov medzi viacerými osobami, napríklad pri cestovaní, bývaní alebo spoločných aktivitách.

1. Analýza Riešenia

Aplikácia je navrhnutá podľa MVVM (Model-View-ViewModel). Tento vzor bol zvolený kvôli čistému oddeleniu zodpovedností, čo uľahčuje testovanie a údržbu aplikácie.

Model (Dátová vrstva):

Súbor: `FirestoreModels.kt`

Obsahuje dátové triedy (`User`, `Group`, `Expense`), ktoré reprezentujú štruktúru dokumentov v databáze Firestore. Tieto triedy sú pasívne – neobsahujú žiadnu logiku, slúžia len ako prepravky na dáta (POJO).

View (Používateľské rozhranie):

Priečinkok: `ui.screens` (obsahuje `AuthScreen`, `GroupsScreen`, `MainScreen`, `AddExpenseScreen`)

Implementované v Jetpack Compose. Obrazovky nepotrebujú prístup do databázy. Iba zobrazujú to, čo im `ViewModel` povie (cez `State`), a reagujú na interakcie používateľa volaním funkcií vo `ViewModel`.

ViewModel (Logická vrstva):

Súbor: `SplitViewModel.kt`

Pôsobí ako riadiaca jednotka aplikácie. Spravuje stav (napr. či je používateľ prihlásený, zoznam skupín) a vykonáva matematické výpočty (bilancie dlhov). Komunikuje s Firebase cez asynchrónne volania (`Coroutines`).

Rozdelenie Balíkov

`com.example.semestralna_praca_vamz`: Hlavná aktivita a vstupný bod

`data.firebase`: Definícia cloudových modelov

`ui.screens`: Jednotlivé obrazovky aplikácie

`ui.theme`: Definícia farieb a štýlov

`ui`: `ViewModel` a pomocné triedy



2. Návrh Riešenia

Možnosti pre používateľa

Akcia	
Registrácia / Prihlásenie	Vstup do systému a ochrana osobných dát.
Správa skupín	Vytvorenie novej skupiny pre výdavky.
Pozvanie člena	Pridanie používateľa do skupiny pomocou e-mailu.
Záznam výdavku	Zadanie sumy, popisu a výber platiteľa.
Voľba typu delenia	Rozhodnutie, či sa suma delí fixne, percentuálne alebo rovnako.
Sledovanie zostatkov	Kontrola dlhov.

1.1 Diagram prípadov použitia (Use Case Diagram)

Diagram pridania výdavku



1.2 Sekvenčný diagram: Pridanie výdavku (Data Flow)

3. Implementácia

Moja aplikácia využíva mnohé znalosti a komponenty, ktoré sme si osvojovali počas semestra. Používa Coroutines pre asynchrónne operácie. Rôzne Jetpack komponenty, ako napríklad Scaffold, LazyColumn a ďalšie. Firestore na online databázu, prepojenie používateľov a prihlasovanie.

Ošetrenie rotácie

V obrazovke AddExpenseScreen sú všetky vstupné polia ošetrené cez rememberSaveable. V kombinácii s verticalScroll na hlavnom kontajneri je zabezpečené, že používateľ môže vyplniť formulár aj v režime Landscape bez straty dát.

Navigácia

Použitý je Navigation Compose s cestami (main/{groupId}), čo umožňuje odkazovať na konkrétne dokumenty v databáze priamo cez URL navigácie.

Sieťová komunikácia a Offline režim

Implementoval som NetworkCallback, ktorý monitoruje stav pripojenia. Aplikácia v reálnom čase upozorňuje používateľa na stratu internetu pomocou dynamického červeného pruhu (Offline indikátor).

Real-time Update

Použitie Firebase listenery (addSnapshotListener) namiesto manuálneho obnovovania (refreshu). To zabezpečuje, že aplikácia pôsobí živo a moderne.

Matematická validácia

Vo ViewModeli a UI je implementovaná kontrola, ktorá nepovolí uloženie výdavku, ak súčet zadaných súm/percent nesúhlasí s celkom (tolerancia 0.01 € kvôli presnosti typu Double).

Bezpečnosť a Exception Handling

Pridal som validáciu vstupných polí pri prihlasovaní a pridávaní členov. Ak používateľ neexistuje alebo sú údaje nesprávne, aplikácia namiesto pádu zobrazí AlertDialog s vysvetlením.



Notifikácie

Trieda NotificationHelper zabezpečuje vytvorenie notifikačného kanála a odosielanie správ o aktuálnom stave dlhov v skupine.

4. Zdroje

Android Developers. (2026). Dostupné na Internete:

<https://developer.android.com/courses/pathways/android-development-with-kotlin-1>

draw.io. (2026). Dostupné na Internete: <https://www.drawio.com/>

Firebase. (2026). Dostupné na Internete: <https://firebase.google.com/docs/guides>

GeeksForGeeks. (2026). Dostupné na Internete: <https://www.geeksforgeeks.org/firebase/firebase-tutorial/>